TokTickIT - Sprint 4 API Specification

Conventions carried over unchanged from Labs 2–3: session auth via the `tt_session` cookie, `requireCsrf` on all state-changing requests, `requireRole(...roles)` for authorization, integer IDs throughout (not UUIDs), route → controller → service structure. Role names below are written as prose (Requester / IT Staff / Administrator) — map them to whatever role-constant casing the existing `middleware/authentication.js` already uses.

Error envelope (assumed consistent with Labs 2–3; align to whatever shape those already return):

```
{ "error": { "code": "STRING_CODE", "message": "human-readable", "field": "optional-field-name"
```

1. Actions Taken

`POST /api/tickets/:ticketId/actions`

Create an Action Taken.

- **Auth:** IT Staff, Administrator (must have access to `:ticketId`).
- **Body:**

  ```
  {
    "actionAt": "2026-09-24T10:15:00Z",
    "description": "Replaced battery, ran diagnostics",
    "resultId": 2,
    "followUpRequired": false,
    "followUpNote": null,
    "attachmentNotes": "see battery_test.png"
  }
  ```

  `actionAt` is optional; when omitted, the database records the current time.
- **201 Response:** full Action Taken record, including server-set `id`, `performedBy` (id + name, from session), `createdAt`, `updatedAt`.
- **Errors:**
  - `400` missing/invalid fields.

- `422` `followUpNote` required when `followUpRequired = true` (BR-04); `description` too short (BR-05); `actionAt` outside the allowed window (BR-06).
- `403` caller is Requester, or IT Staff without access to the Ticket.
- `404` Ticket not found.
- `409` Ticket status is `Closed` or `Cancelled` (BR-16).

`GET /api/tickets/:ticketId/actions`

List Actions Taken for a Ticket, ordered by `actionAt` descending.

- **Auth:** IT Staff, Administrator (with access), or the owning Requester (read-only).
- **200 Response:** array of Action Taken records (same shape as create response), each including the resolved `result` name and `performedBy` display name (not just ids).
- **Errors:** `403` Requester requesting a Ticket they don't own; `404` Ticket not found.

`PATCH /api/tickets/:ticketId/actions/:actionId`

Edit an existing Action Taken.

- **Auth:** IT Staff, Administrator (with access to the Ticket) — any such user, not only the original performer (BR-15).
- **Body:** any subset of `description`, `resultId`, `followUpRequired`, `followUpNote`, `attachmentNotes`, plus the required concurrency token:

```
{ "description": "Updated notes", "updatedAt": "2026-09-24T10:15:00Z" }
```

- **200 Response:** updated record.
- **Errors:** `400` / `422` as above; `403` forbidden; `404` not found; `409 Conflict` when `updatedAt` doesn't match the current record — response includes the current record so the client can refetch/merge (BR-11).

`GET /api/reference/action-results`

List Result reference values (mirrors the existing `GET /api/categories` pattern).

- **Auth:** any authenticated role.
- **200 Response:** `[{ "id": 1, "name": "In Progress", "isActive": true }, ...]`.

2. Ticket status transitions

`PATCH /api/tickets/:ticketId/status`

General status transition, matrix-enforced (`specification.md` §5.2). Handles every transition **except** the move to `Resolved` , which stays on the Lab 3 endpoint below so its existing review/audit behavior is preserved.

- **Auth:** varies by transition — see the matrix; the backend re-checks the caller's role for the specific `from → to` pair regardless of what the client requested.
- **Body:** `{ "toStatus": "In Progress", "updatedAt": "2026-09-24T10:15:00Z" }`
- **200 Response:** updated Ticket summary (id, status, owner, timestamps).
- **Errors:** `400` unknown status value; `403` role not permitted for this transition (e.g. non-Administrator attempting to exit `Cancelled` / `Closed`); `409` transition not in the matrix, or stale `updatedAt` .

`PATCH /api/tickets/:ticketId/resolution` (existing, Lab 3 — extended)

Unchanged request/response shape from Lab 3. Now additionally enforces BR-07: rejects with `409` and an explanatory `message` if the Ticket has no Action Taken record with `result.name = "Resolved"` recorded by IT Staff/Administrator.

3. Dashboards

Dashboard endpoints return concise, pre-aggregated data — never raw Ticket collections — per the handout's dashboard contract requirement. Each metric includes a `filters` object the client uses to build the corresponding Ticket Queue/My Tickets drill-down link.

`GET /api/requester/dashboard`

- **Auth:** Requester. Server derives the Requester from the session; any client-supplied requester/user id is ignored (BR-14).
- **200 Response:**

```
{
  "totalOpen": 3,
  "waitingForYou": 1,
  "recentlyResolved": 5,
  "closed": 12,
  "recentTickets": [ { "id": 1234, "title": "...", "status": "In Progress", "updatedAt": ".
}
```


`recentTickets` capped at 5.

`GET /api/staff/dashboard`

- **Auth:** IT Staff, Administrator.
- **200 Response:**


```
{
  "unassigned": 14,
  "myAssigned": 16,
  "new": 14,
  "open": 23,
  "inProgress": 18,
  "waitingForRequester": 7,
  "byPriority": [ { "priorityId": 1, "priorityName": "Urgent", "count": 4 } ],
  "recentTickets": [ { "id": 1234, "title": "...", "status": "Open", "updatedAt": "..." } ]
}
```


`GET /api/admin/dashboard`

- **Auth:** Administrator only.
- **200 Response:** the staff dashboard payload above, plus:


```
{ "userCounts": { "total": 42, "active": 39, "byRole": [ { "role": "IT Staff", "count": 10
```


Empty behavior (all three): zero-count fields return `0`, `recentTickets` returns `[]` — never an error, never a missing field (AC-09, AC-11).

4. Unchanged from Labs 2–3

Auth (`/api/auth/*`), Tickets CRUD, `/api/tickets/:id/comments`, `/api/tickets/:id/notes`, `/api/tickets/:id/attachments`, `/api/categories`, `/api/users` (Administrator). No request or

response shape for these changes in Lab 4.

5. Authorization matrix

Endpoint	Requester	IT Staff	Administrator
`POST .../actions`	×	✓ (with access)	✓
`GET .../actions`	✓ (own Ticket only)	✓ (with access)	✓
`PATCH .../actions/:id`	×	✓ (with access)	✓
`GET /reference/action-results`	✓	✓	✓
`PATCH .../status` (most transitions)	×	✓	✓
`PATCH .../status` (New→Cancelled, Resolved→Reopened)	✓ (own Ticket)	✓	✓
`PATCH .../status` (exit Cancelled/Closed)	×	×	✓
`PATCH .../resolution`	×	✓	✓
`GET /dashboard/requester`	✓ (own data)	×	×
`GET /dashboard/staff`	×	✓	✓
`GET /dashboard/admin`	×	×	✓

Every row above is enforced in the backend controller/middleware layer — hiding a control in the client is never treated as authorization.

6. Status codes used

Lab 4 follows the Lab 3 status-code conventions unchanged. Only the rows below are extended.

Status	Lab 4 usage
`200`	Action Taken listed or edited; status transition succeeded; dashboard retrieved

Status	Lab 4 usage
`201`	Action Taken created (added to the Lab 3 "created" list)
`400`	Any validation failure, including: Description too short (BR-05), Follow-up Note missing when Follow-Up Required is `true` (BR-04), Action Date/Time out of range (BR-06, `code: INVALID_ACTION_TIME`), invalid or inactive `resultId`, invalid `actionId`. Field-level errors return `error.field` and `error.fieldErrors`.
`403`	Requester calls a create/edit Action Taken endpoint; IT Staff attempts to reopen a `Cancelled` / `Closed` Ticket (BR-10); missing CSRF header
`404`	Ticket or Action Taken not found, or Ticket not accessible to the caller
`409`	Create on a `Closed` / `Cancelled` Ticket (BR-16, `INVALID_STATUS`); stale `updatedAt` (BR-11, `CONFLICT`, current record returned); transition not in the matrix or Resolved gate unmet (BR-07/BR-09)
`422`	Not used by Lab 4. Remains reserved for the Lab 3 Attachment-limit rule.
`500`	Unexpected server error; generic safe message only